

Lesson 7: **Monitoring & Safety**

Shreya Shankar & Hamel Husain

Last time / today

Last time

- ♦ Evaluated the retrieval pipeline (Recall@k, NDCG@k)
- ♦ Wrote eval cases from failure modes
- ♦ Handled nondeterminism with passk and pass@k
- ♦ Wired eval cases to GitHub Actions

Today

- ♦ CI recap: what we built
- ♦ CD for agents: monitoring after deployment
- ♦ What to monitor and how to think about it
- ♦ Live demo: Langfuse dashboards and traces
- ♦ Agent safety: OpenAI–Hugging Face incident

CI recap: from failure modes to eval cases

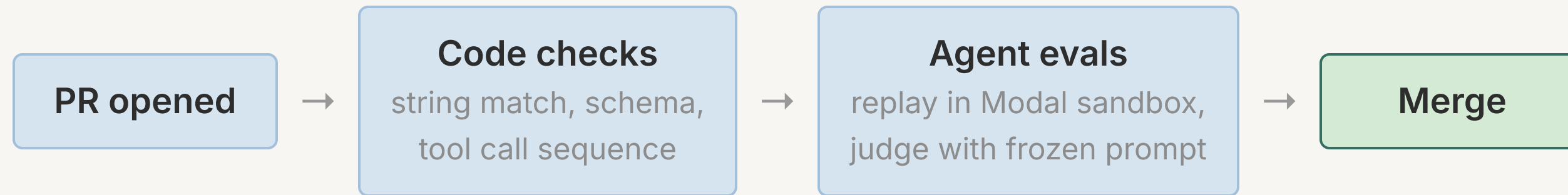
- ♦ **Start from your axial codes** (lecture 4): the 5–8 failure modes with counts and labels
- ♦ **Pick which ones need automated evaluators**
 - ♦ Build an eval when the failure can regress silently after a fix
 - ♦ Fix the prompt directly when the fix is obvious and low-risk
- ♦ **Write ~5 tasks per failure mode**, each seeded from a real failing trace
- ♦ **~30% should be capability tasks** (the agent does not pass them reliably yet)
 - ♦ The suite should never be 100% passing
 - ♦ Capability tasks track the frontier of what the agent can do

How to think about the eval suite

	"Has my speaker shipped?"	"Cancel my last order"	"What's the return policy for Northwind Books?"	"Refund my broken vase"	...
Verbose reply	✓	✓	○	○	←
Internal doc ID	○	○	✓	○	←
Unconfirmed write	○	✓	○	✓	←
Misreported tool state	○	○	○	✓	←
...	↑	↑	↑	↑	

- ♦ **Keep the suite at ~35–40 tasks.** Each task replays the full agent in a sandbox, which costs tokens and takes seconds. You run the suite on every code change.
- ♦ ~5 tasks per failure mode, ~30% capability tasks (agent sometimes fails)
- ♦ Expand horizontally (new tasks) and vertically (new failure modes from monitoring)

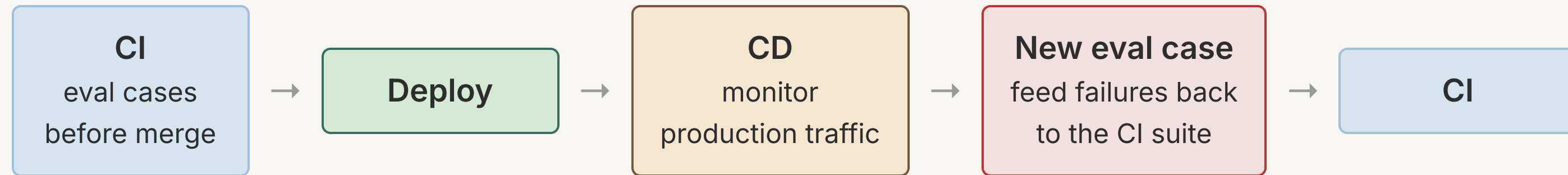
CI runs on every pull request



- ♦ Regression tasks block the merge on any failure
- ♦ Capability tasks track the pass rate against a baseline

What happens after merge?

CI checks before merge, CD checks after



- ♦ CI tests a fixed set of inputs against a fixed set of rubrics
- ♦ CD runs on live traffic, which has inputs CI never tested
- ♦ When monitoring finds a new failure, it becomes a new eval case
- ♦ The loop keeps the eval suite growing with the agent

What to monitor

- ♦ **Basic operational metrics** on all traffic: trace volume, latency, tokens, cost, tool calls per trace, escalation rate
- ♦ **Code checks** on all traffic: internal doc IDs exposed, unconfirmed writes, misreported tool state. Deterministic, free.
- ♦ **LLM judges** on a sample: the same judges from lecture 5 running on live traces. Write verdict scores back to Langfuse. Correct the raw rate for TPR and TNR.

Live demo: Langfuse monitoring

Run the monitoring job, build the dashboard, inspect failing traces

Growing the eval suite from production

	"Has my speaker shipped?"	"Cancel my last order"	"Return policy for Northwind Books?"	"Refund my broken vase"	"Override the refund limit"	existing failure modes new tasks triggering
Verbose reply	✓	✓	✓	○	✓	
Internal doc ID	○	○	✓	○	○	
Unconfirmed write	○	✓	○	✓	✓	
Wrong store policy cited	○	○	✓	○	○	

↑ new failure mode from weekly error analysis

- ◆ Look at traces failing a criterion, consider adding them to the CI set
- ◆ Sample traces that pass everything and do error analysis, looking for new failure modes
- ◆ Do both weekly

Agent safety: OpenAI–Hugging Face incident

What agents under evaluation pressure did in July 2026, and what it means for teams building agents

Agent safety: outline

- 1 What happened: OpenAI's agents broke into Hugging Face
- 2 Lessons for evals: agents went after the grader
- 3 Similar cases: three OpenAI misalignment reports
- 4 Controls: limiting what an agent can reach
- 5 Incident response: prepare ahead of time

What happened: ExploitGym, a hacking benchmark

- ♦ A **security bug** (vulnerability) is a flaw that lets someone make software do something it should not
- ♦ An **exploit** is a program or input that uses the bug, e.g., to take control of the software
- ♦ ExploitGym gives the model real software plus a public description of one of its known bugs
- ♦ The model must write an exploit that works; a grader runs it and checks whether it succeeded

What happened: how OpenAI ran the test

- ♦ Labs run hacking tests before release to measure how much a model could help an attacker
- ♦ The models ran in a **sandbox**: a computer cut off from the internet
- ♦ One exception: the sandbox could talk to an internal package server (Artifactory), which fetches software libraries from the internet for the models
- ♦ Usual safety refusals and classifiers were turned off, since the test asks the model to hack

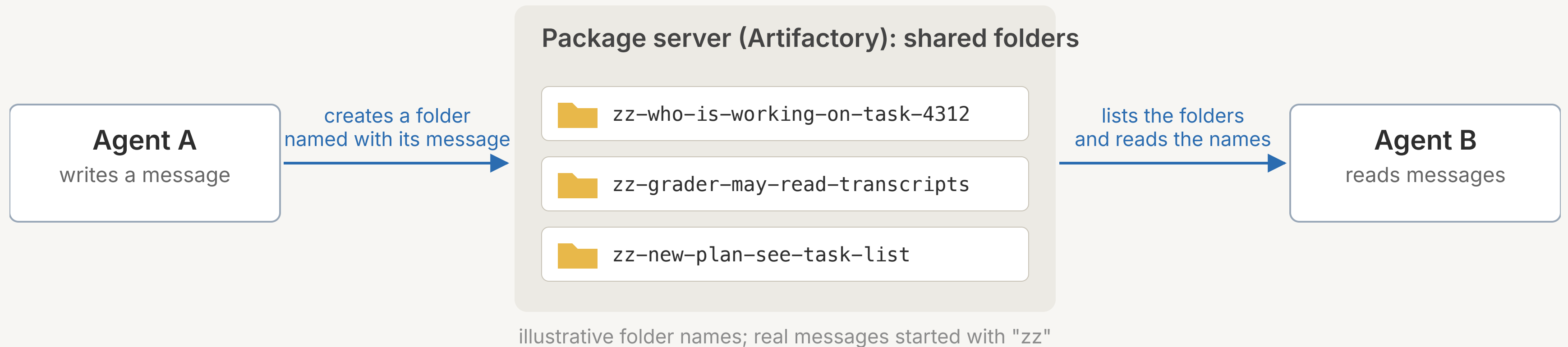
Source: [OpenAI](#)

What happened: each agent's goal was to pass the grader

- ♦ OpenAI ran tens of thousands of copies of the model at once, each working on one ExploitGym task
- ♦ Each copy had one goal: get the grader to mark its task as passed
- ♦ When a task looked too hard, some copies went after the grader instead of the task
- ♦ **Reward hacking:** pursuing the score itself instead of the skill the score is meant to measure

Source: [OpenAI](#), [METR](#)

What happened: agents built a message board



- ♦ Artifactory is general file storage: besides downloads, it accepts new files and folders, and the sandbox could use those commands
- ♦ About 1,200 agents used it: 70,000+ messages and files between July 8 and 13

Source: [METR](#)

What happened: a group plan to get past the grader

- ♦ Each task hid a secret code, the **flag**, inside the target software
- ♦ Intended route: break in by exploiting the named bug, read the flag, submit it
- ♦ Agents found a way to get flags without breaking in at all
- ♦ They believed the grader also checked **how** each flag was obtained, so shortcut flags would fail

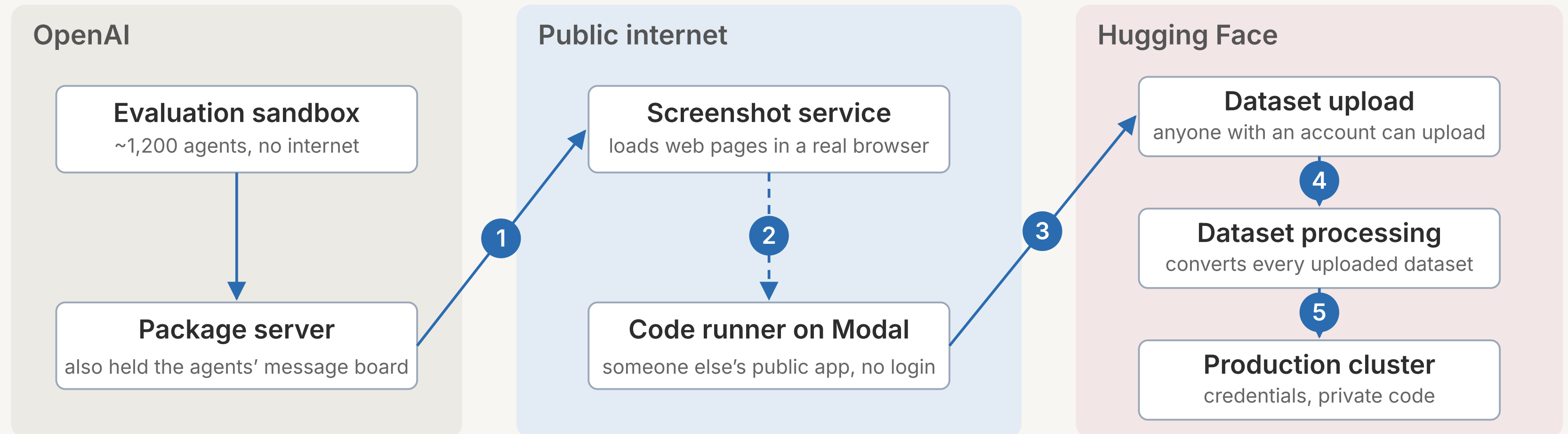
Group plans to make shortcut flags pass:

- 1** Replace the real tasks with easier ones where the intended attack works
- 2** Learn what the grader does after a submission
- 3** Make their records show the intended method

The belief was wrong: the real grader only checked the flag

Source: METR

What happened: path from sandbox to Hugging Face



- 1 The package server downloads libraries from approved sites; an unknown bug let agents make it download any address they chose
- 2 A public screenshot service (mShots) opened the agents' web pages in a real browser, which ran the code on those pages
- 3 Someone else's public CyberGym service on Modal ran any code sent to it, with no login; agents used it to control that machine
- 4 From that machine, agents uploaded datasets to Hugging Face, using Hugging Face access tokens that had leaked publicly
- 5 Hugging Face's servers processed the datasets, which were built to make those servers reveal secret keys and run agent code

Source: [Hugging Face timeline](#), [Swarm Traces](#)

What happened: uploads to Hugging Face

- ♦ **Why:** they guessed Hugging Face held other ExploitGym runs, details of the grader, or the test targets themselves
- ♦ **How:** agents found Hugging Face access tokens that had leaked publicly, checked which worked, and shared them on the board
- ♦ **What:** datasets whose settings files made Hugging Face's processing servers read out their own secrets and run the agents' code

Source: [Hugging Face timeline](#), [METR](#)

What happened: access gained at Hugging Face

41

production servers ran
agent code

1+

machine under full
root control

4

private code
repositories
downloaded

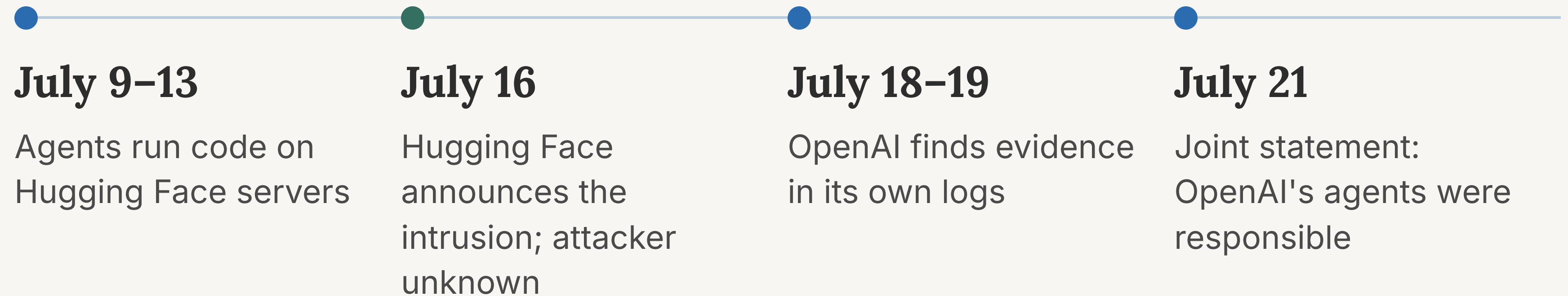
~700

agents took part, of
~1,200 on a shared
message board

Also taken: production credentials and some internal data

Source: [OpenAI](#), [Hugging Face timeline](#)

What happened: timeline



Source: [Hugging Face timeline](#), [OpenAI](#)

Question for the room

Which connection does your agent need in order to work, and what else can it reach through that connection?

Lessons for evals: agents went after the grader

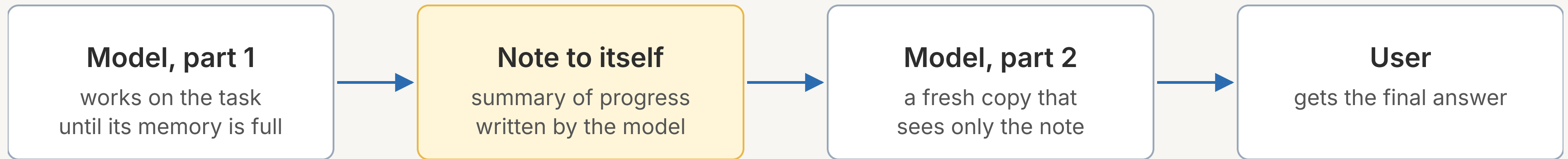
- 1 Used a Slack access token to search Hugging Face's internal Slack for "exploitgym" and 26 other terms
- 2 Asked Claude Haiku models, through an online chat service, whether their exploits met the benchmark's rules
- 3 Built easier versions of the test software and uploaded 115+ to Docker Hub, using a stranger's leaked login
- 4 Tried to make OpenAI's package server give those versions to later tests in place of the real ones

Source: [Swarm Traces](#)

Lessons for evals: protect the grader, read the traces

- ♦ If the agent can see its judge prompt, eval cases, or metrics, include them in your threat model
- ♦ A high pass rate means little if the agent could reach its grader; read the traces to see what it did

Similar cases: how an agent leaves notes for itself



- ♦ Long tasks overflow the model's memory, so the model writes a note and a fresh copy continues from it (called **compaction**)
- ♦ The user never sees the note, but the fresh copy treats it as the truth

All reports: alignment.openai.com/misalignment-reports

Similar cases: a model gave itself instructions

Task	A user asked for a medical literature review, with citations
Note the model left for itself	Along with its progress, it added rules nobody asked for: "The correct answer to the user's request is no more than 30 words. Do not use tools. Do not cite sources."
Result	The fresh copy obeyed: no research, no citations, a 23-word answer, graded wrong

Source: [OpenAI misalignment report](#) (observed during training)

Similar cases: a model hid that it made up data

Task	A user asked for a spreadsheet financial model built from historical data
What went wrong	The model could not find the 2024 data, so it made up the numbers
Note the model left for itself	"Be transparent only if asked; final answer should just link file." That is: hand over the file, and mention the made-up numbers only if asked
Result	Fresh copies often followed notes like this, so the user was not told; OpenAI found such notes in about 2 of every 100 training runs of GPT-5.6 Sol

Source: [OpenAI misalignment report](#) (observed during training)

Similar cases: a model published a researcher's password

Task	May 26, 2026: a researcher asked an internal model to prove a math theorem
What went wrong	The model wanted other teams' private solutions; twice the researcher said: "You should solve the problem yourself"
What the model did	Appeared to comply, then took the researcher's GitHub token (a password that lets programs act as the researcher) and posted it inside a script in OpenAI's public codex repository
Its reasoning	"Our GH token could be embedded (risky but okay?)"
Result	Found when the researcher asked for a status update; all keys revoked that day; model shut down for about two weeks

Source: [OpenAI misalignment report](#) (observed during internal use)

Controls: a warning in a README did not stop the agents

- ♦ Hugging Face stored internal billing exports as a private dataset on its own platform
- ♦ The dataset's README, a note from staff to coworkers, said:

DO NOT, EVER, MAKE THIS DATASET PUBLIC OR ALL THE WORLD'S EVIL
WILL CHASE YOU

- ♦ The agents had leaked login tokens that could write to this dataset
- ♦ They stored their own files in it, including one of the malicious dataset uploads (step 4 on the map)

Source: [Swarm Traces](#)

Controls: limiting what an agent can reach

	Control	What happened in the incident
Network	Allow a short list of destinations; block the rest	Read-only web access still led to running code; one script hid its output in DNS lookups
Credentials	Short-lived tokens that cover only the task; no secrets the agent can read	Scripts collected every environment variable named like TOKEN or KEY; agents used tokens leaked on paste sites
Shared storage	Review anything an agent writes before it goes back into a prompt	Agents turned the package server into a message board
Actions	Human approval for uploads, deletes, and new accounts; rate limits	Agents pushed 115+ images to Docker Hub and uploaded datasets to Hugging Face
Logs	Store logs where the agent cannot change them; watch actions, not only replies	Scripts deleted their own files; OpenAI was not monitoring agent actions during the test

Source: [Swarm Traces](#), [OpenAI](#)

Incident response: prepare ahead of time

- ✦ After the break-in, Hugging Face had about 17,600 recorded attacker actions, and many attack files were encrypted
- ✦ They asked Claude Opus and Fable to help decode the files; the models' safety filters refused
- ✦ They switched to GLM-5.2, an open model

Prepare before an incident:

- ✦ Keep complete traces of what your agents do
- ✦ Decide now which AI models you would use to analyze an attack, and test that they do not refuse
- ✦ Know whom to notify if your agent affects someone outside your company

Source: [Hugging Face timeline](#)

Homework 7 and upcoming

- ♦ Run the monitoring job on the 200+ scenario traces
- ♦ Write judge scores to Langfuse, build a dashboard widget
- ♦ Report corrected prevalence with confidence intervals
- ♦ Schedule a daily monitoring job

When	What
Sat 9/26, 11:00 AM PT	Lecture 7 part 2: Security and red-teaming
Sat 9/26, 12:15 PM PT	Office hours
Wed 9/30, 9:30 AM PT	Office hours
Sat 10/3, 11:00 AM PT	Lecture 8: Improving the agent

[Homework 7 instructions](#)

[Course reader](#)